

Question 1:

I. Architectural Approach

The system is designed using a Microservices Architecture combined with an Event-driven model. This is the best solution to ensure each part stays separate, scales easily, and remains stable under high traffic.

II. Component Analysis

1. User Interfaces and User Groups

The system provides two specialized interfaces:

- Mobile App: For customers. It focuses on ease of use for browsing events, choosing seats visually, making quick payments, and managing ticket history.
- Desktop System: For staff and managers. It provides powerful tools to update event content, design complex seating layouts, track sales reports, and manage the system.

2. Coordination, Security, and Traffic Management

To handle huge traffic and keep data safe:

- Single Entry Point (API Gateway): The only entrance for all requests. It intelligently routes each request to the correct service.
- Security Layer (WAF/Firewall): Acts as a "shield" to block unauthorized access and cyber-attacks, ensuring only valid requests enter the system.
- Load Balancer: Automatically distributes work across servers. This prevents the system from "clogging" when many people book tickets at the same time.

3. Independent Functional Services (Backend Microservices)

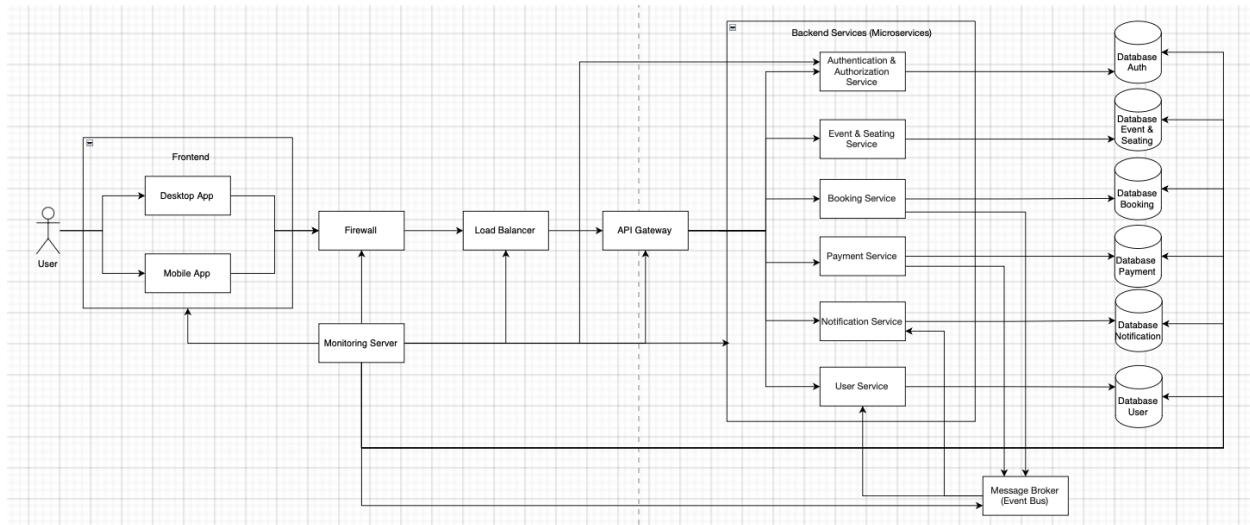
The system is broken down into separate parts for easy management and upgrades:

- Auth Service: Manages logins, security, and access levels.
- Event & Seating Service: Manages all event details and seat locations.
- Booking Service: Handles the logic for reserving seats and checking ticket availability in real-time.
- User Service: Stores personal information and transaction history.
- Payment Service: Connects to payment gateways to process financial transactions.
- Notification Service: Automatically sends emails or messages for reminders and booking confirmations.

4. Data Isolation, Interaction, and Consistency

- Database per Service: Each service has its own storage. Maintaining one service (e.g., Payments) will not affect the others.
- Event-driven Communication: Uses a Message Broker so services can talk to each other without being directly dependent.
- Data Consistency (Eventual Consistency): Since data is stored separately, the system uses special patterns. If an error occurs (e.g., payment is successful but the ticket fails to book), the system automatically triggers a refund or fix to protect the customer.
- Health Monitoring: Continuously tracks every part of the system to detect issues early and keep it running 24/7.

III. High-level block diagram



Description:

- **Frontend:** The frontend is the interface through which users interact with the ticketing system. It consists of two main parts:
 - **The Desktop App:** This is used by internal staff and managers to update event information, arrange seating layouts, track ticket sales, and manage users.
 - **The Mobile App:** Designed for customers to browse ongoing events, choose seats, purchase tickets, make payments, and review their previous purchase history.
- **Firewall:** Acts as a protection layer (DMZ) to block invalid external access and protect the system from unauthorized threats by filtering incoming and outgoing traffic.
- **Load Balancer:** Automatically distributes incoming network traffic across multiple processing nodes to prevent overload during peak times, ensuring high system availability and reliability.
- **API Gateway:** The single entry point for all client requests. It receives all user requests and automatically routes them to the appropriate backend microservices for processing.
- **Backend Services (Microservices):** The system is divided into dedicated functional components to make management and expansion easier:
 - **Authentication & Authorization Service:** Manages user identity, login processes, and access control to ensure system security.
 - **Event & Seating Service:** Handles event content, management of event information, and seating layouts.
 - **Booking Service:** Manages the core logic of ticket booking, seat selection, and tracking ticket sales.
 - **Payment Service:** Dedicated component for processing financial transactions and ticket purchases.
 - **Notification Service:** Responsible for sending alerts to customers after successful purchases or when an event is about to start.
 - **User Service:** Manages profile information and history for both customers and internal staff.
- **Database:** Following the "Database per Service" pattern, each functional component has its own dedicated database (e.g., Database Auth, Database Event, etc.). This ensures data isolation, so upgrading or maintaining one part does not interrupt the others.
- **Message Broker (Event Bus):** Facilitates asynchronous communication between services, such as triggering a notification immediately after a payment is processed.
- **Monitoring Server:** Continuously monitors the overall health and performance of all servers, services, and databases to quickly detect issues and evaluate system stability.

Question 2:

1. Identified Code Smells

Based on the initial source code, there are at least three significant design flaws:

- **Meaningful Naming:** The class name `SystemManager` and the method name `DoEverything` are too unclear. They do not accurately reflect the specific business logic of processing payments and sending notifications.
- **Violation of Single Responsibility Principle (SRP):** The `DoEverything` method handles too many tasks: checking the payment type, processing the payment logic, and deciding which type of email to send.
- **Magic Strings:** The values "card" and "paypal" are hardcoded directly for comparison. It's easy to make typing mistakes and makes it difficult to manage or extend payment methods in the future.
- **Tight Coupling:** The payment logic is strictly tied to an if-else structure. Every time a new payment method is added, the existing source code must be modified.

2. Refactored Code

The code has been restructured to ensure separation of concerns and utilize polymorphism:

```
using System;

// Define a common interface for payment methods to reduce coupling
public interface IPaymentMethod
{
    void Process(double amount);
}

// Concrete class for Card payments
public class CardPayment : IPaymentMethod
{
    public void Process(double amount) => Console.WriteLine($"Pay with card: {amount}");
}

// Concrete class for Paypal payments
public class PaypalPayment : IPaymentMethod
{
    public void Process(double amount) => Console.WriteLine($"Pay with paypal: {amount}");
}

// Specialized class for notifications (Ensures Single Responsibility Principle)
public class NotificationService
{
    private const double VipThreshold = 500; // Remove Magic Number with a constant

    public void SendEmail(double amount)
    {
        // Use conditional operator for cleaner code
        string emailType = (amount > VipThreshold) ? "VIP" : "normal";
        Console.WriteLine($"Send {emailType} email");
    }
}

// Main coordinator class with a meaningful name
public class PaymentProcessor
{
    private readonly NotificationService _notificationService;

    public PaymentProcessor(NotificationService notificationService)
    {
        _notificationService = notificationService;
    }

    // Accepts an Interface to reduce dependency
    public void Execute(IPaymentMethod paymentMethod, double amount)
    {
        // Process payment without knowing the underlying implementation details
    }
}
```

```

        paymentMethod.Process(amount);

        // Delegate email tasks to the notification service
        _notificationService.SendEmail(amount);
    }
}

```

3. Explanations of Refactoring Decisions

- **Meaningful Naming:** Renaming SystemManager to PaymentProcessor and DoEverything to Execute makes the code easy to understand.
- **Separate Responsibilities:** Moving the email logic to NotificationService and payment logic into specific classes implementing IPaymentMethod ensures that each component focuses on a single responsibility.
- **Remove Magic Strings:** Hardcoded strings like "card" and "paypal" are replaced by concrete objects. The number 500 is replaced by the constant VipThreshold, making it easier to change the VIP criteria in the future.
- **Reduce Coupling:** By applying the Strategy Pattern (via IPaymentMethod), the PaymentProcessor class no longer depends on specific payment types. New payment methods can be added by creating new classes without modifying the existing code.

Question 3:

```

using System;

public interface IPaymentMethod
{
    void ProcessPayment(double amount);
}

public class CardPayment : IPaymentMethod
{
    public void ProcessPayment(double amount) =>
    {
        Console.WriteLine($"Paid {amount} using Credit Card.");
    }
}

public class PaypalPayment : IPaymentMethod
{
    public void ProcessPayment(double amount) =>
    {
        Console.WriteLine($"Paid {amount} using PayPal.");
    }
}

public class MomoPayment : IPaymentMethod
{
    public void ProcessPayment(double amount) =>
    {
        Console.WriteLine($"Paid {amount} using Momo Wallet.");
    }
}

```

```

public class ZaloPayPayment : IPaymentMethod
{
    public void ProcessPayment(double amount) =>
        Console.WriteLine($"Paid {amount} using ZaloPay.");
}

public class PaymentProcessor
{
    public void Execute(IPaymentMethod paymentMethod, double amount)
    {
        if (paymentMethod == null) return;
        paymentMethod.ProcessPayment(amount);
    }
}

public class Program
{
    public static void Main()
    {
        PaymentProcessor processor = new PaymentProcessor();

        processor.Execute (new CardPayment(), 100.0);
        processor.Execute (new MomoPayment(), 250.0);
        processor.Execute (new ZaloPayPayment(), 300.0);
    }
}

```

Nếu tôi muốn một đối tượng `PaymentProcessor` sinh ra chỉ để chuyên phục vụ cho một phương thức thanh toán cố định thì làm sao?
(Constructor Injection)

```

public class PaymentProcessor
{
    private readonly IPaymentMethod _method;

    // Truyền vào ngay từ khi "đẻ" ra đối tượng
    public PaymentProcessor(IPaymentMethod method)
    {
        _method = method;
    }
}

```

```

    {
        _method = method;
    }

    public void Execute(double amount)
    {
        _method.ProcessPayment(amount);
    }
}

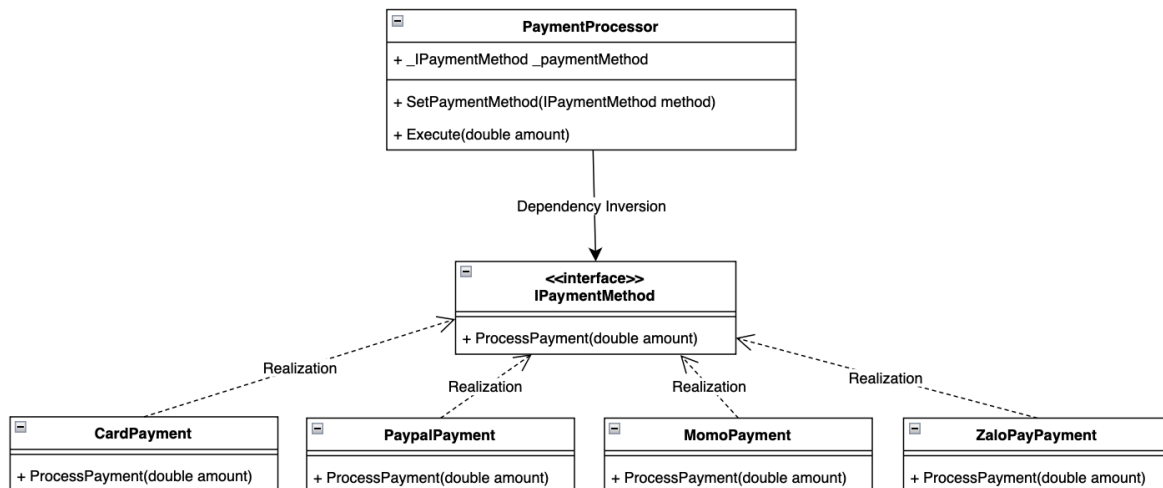
// Cách dùng ở Main:

var cardProcessor = new PaymentProcessor(new CardPayment());

cardProcessor.Execute(100.0);

```

Diagram:



Description:

The class diagram illustrates the application of the Strategy Pattern to decouple payment processing logic from specific payment methods. This structure consists of three main components:

1. Abstraction (The Interface)

- **IPaymentMethod:** This is an interface that acts as a common "contract." It only defines a single method: `ProcessPayment(double amount)`.
- **Role:** This is the key to implementing the Dependency Inversion Principle. Instead of depending on concrete classes, the system depends on this abstract interface.

2. Concrete Realizations

- These include classes such as CardPayment, PaypalPayment, MomoPayment, and ZaloPayPayment.
- Relationship: These classes implement (Realization) the IPaymentMethod interface. Each class defines its own specific logic for its respective payment type.
- Role: This demonstrates the Open/Closed Principle. When a new payment method (e.g., Apple Pay) needs to be added, one simply creates a new class implementing the interface without modifying any existing source code.

3. Context Class

- PaymentProcessor: Connects to IPaymentMethod through a Dependency relationship.
- Structure: It contains an Execute method that accepts IPaymentMethod as a parameter to process payments flexibly without storing any state.
- Relationship: This class connects to the Interface through a Dependency Inversion relationship, achieving Loose Coupling by removing the need for a SetPaymentMethod or a private property.

Question 4:

1. Folder Structure

The project is organized to ensure a clear separation of concerns:

- Interfaces/: Defines INotifier, providing the abstraction layer for the entire system.
- Models/: Contains concrete strategy implementations (Email, SMS, Push).
- Factory/: Centralizes object creation logic in a single location.
- Decorators/: Contains additional features (Logging, Retry) that run alongside the core logic.
- Program.cs: Acts as the entry point to coordinate and combine all components.

2. Design Decisions

The system implements three core design patterns:

- Strategy Pattern: Utilizes the INotifier interface to easily swap between delivery methods (Email, SMS, Push) without affecting the main processing logic.
- Factory Pattern: The NotifierFactory hides complex instantiation logic. The client simply requests a notification type without needing to know how it is created.
- Decorator Pattern: Allows extending functionality, such as logging or retrying, by "wrapping" the original object. This follows the Open/Closed Principle: new features are added without modifying existing class code.

3. Key Benefits

- Scalability: Adding a new notification type only requires creating a new class in the Models folder.
- Maintainability: Each class has a single responsibility (SRP), making the code easier to debug.
- Flexibility: Multiple features (e.g., both Logging and Retry) can be dynamically combined for a single object at runtime.